

Week 03: Higher-order functions and folds

Functional Programming in Lean

Learning goals

- ▶ Use a function as an argument
- ▶ Write polymorphic list operations
- ▶ Express a traversal as a fold

Central idea

map changes each element. A fold replaces constructors with operations.

Lean example

```
def foldRight (f :  $\alpha \rightarrow \beta \rightarrow \beta$ )  
  (base :  $\beta$ ) : List  $\alpha \rightarrow \beta$   
| [] => base  
| x :: xs => f x (foldRight f base xs)
```

What to notice

- ▶ α and β may be different types
- ▶ Currying makes partial application possible
- ▶ The fold result type need not match the element type

Live coding

1. Write `mapList` by recursion
2. Complete `foldRight`
3. Define `sum` with `foldRight`

Pause and predict

What is the type of `foldRight (· + ·) 0`?

Exercise connection

Open `exercises/Week03.md` after the lecture.

The complete example is `Course/Week03.lean`; the fill-in version is `Templates/Week03.lean`.